QUALIDADE E TESTES DE SOFTWARE · 6º DSM

Testes Não Funcionais: Testes de Segurança

Security Testing aplicado à NotasAPI, uma API REST de notas acadêmicas

Eduardo Fontes · Eduardo Kuwahara · Fábio Hiroshi · João Rossi
Kathellyn Caroline · Paulo Henrique · Victor Daniel

Prof. Lucas Nadalete · FATEC São José dos Campos · 26/11/2026



ROTEIRO

O que vamos apresentar

01
Introdução

02
Histórico

03
O que é?

04
Qual o propósito?

05
Vantagens e desvantagens

06
Ferramentas e frameworks

07
Ilustração do exemplo

08
Demonstração e resultados

09
Conclusão e referências

Por que testar a segurança do software?



APIs são a porta de entrada

Apps web e móveis conversam com o servidor por APIs. Os ataques a elas ganharam uma lista própria da OWASP em 2019.



Corrigir tarde custa caro

Falha achada em produção exige correção às pressas, novo deploy, resposta a incidente e aviso aos afetados.



Dados pessoais têm proteção legal

Notas e cadastros de alunos são dados pessoais protegidos pela LGPD (Lei nº 13.709/2018).

Onde o tema se encaixa

Testes funcionais

O que o sistema faz: o professor consegue lançar uma nota?

Testes não funcionais

Como o sistema se comporta: desempenho, usabilidade, confiabilidade e **segurança**.

Segurança na ISO/IEC 25010:2023

Subcaracterística	Pergunta que o teste responde
Confidencialidade	Só quem pode acessa o dado?
Integridade	O dado é alterado só por quem pode?
Não repúdio	Dá para provar que a ação ocorreu?
Responsabilização	A ação é rastreável a quem a fez?
Autenticidade	O usuário é quem diz ser?
Resistência	O sistema suporta um ataque?

O exemplo prático: NotasAPI

API REST de uma escola técnica fictícia: professores lançam notas e alunos consultam o próprio desempenho.

Stack: Python, FastAPI, JWT (PyJWT), SQLite + SQLAlchemy, Docker

Papéis: ALUNO e PROFESSOR

vulneravel: falhas propositais

main: versão corrigida

Rota	Função	Quem acessa
`POST /auth/registro`	Cadastra aluno	Público
`POST /auth/login`	Devolve token JWT	Público
`GET /notas/minhas`	Notas do aluno	Aluno
`GET /notas/{id}`	Detalha uma nota	Dono ou professor
`POST /notas`	Lança nota	Professor
`GET /alunos?nome=`	Busca alunos	Autenticado
`PATCH /usuarios/me`	Edita nome e e-mail	Autenticado

Dos tiger teams ao DevSecOps

Anos 1970	Relatório de Anderson (1972) para a Força Aérea dos EUA e os tiger teams: nasce o teste de invasão como prática formal.
1988	Worm de Morris, primeiro grande incidente da Internet: explorou estouro de buffer e senhas fracas (SPAFFORD, 1989).
Anos 1990	A Web se populariza e amplia a superfície de ataque; surgem scanners automatizados de vulnerabilidades (SATAN, Nessus).
2001–2004	Nasce a OWASP (2001) e sai o primeiro OWASP Top 10 (2003); testes de segurança baseados em risco (POTTER; MCGRAW, 2004).
2006	Security Development Lifecycle da Microsoft: segurança em todas as fases do desenvolvimento (HOWARD; LIPNER, 2006).
2008–2010s	Guia NIST SP 800-115 (SCARFONE et al., 2008); SAST, DAST e SCA se consolidam; movimento DevSecOps e shift-left.
2019–hoje	OWASP API Security Top 10 (2019 e 2023) e OWASP Top 10 para aplicações com LLM: APIs e IA generativa no centro.

Testes de segurança são técnicas que verificam se o software **protege dados e funcionalidades** como deveria e se **resiste a usos maliciosos**, encontrando vulnerabilidades antes que sejam exploradas.

Com base em Felderer et al. (2016) e Potter e McGraw (2004)

Duas perguntas diferentes



TESTE FUNCIONAL DE SEGURANÇA

O controle existe e funciona?

Rota protegida chamada sem token devolve 401.

```
CT01 · sem token → 401
```



TESTE BASEADO EM RISCO

O atacante consegue contornar?

Token forjado com outro segredo, com alg none ou já expirado também tem de ser recusado.

```
CT02–CT04 · token forjado → 401
```

Na branch vulneravel: o CT01 passa e o CT02–CT04 falham. O controle existe, mas é contornável.

Principais tipos e técnicas

Técnica	O que faz	Caixa	Quando
SAST	Analisa o código-fonte sem executá-lo	Branca	A cada commit
SCA	Procura CVEs nas dependências	Branca	A cada build
Varredura de segredos	Acha chaves e senhas no repositório	Branca	A cada commit
Revisão de código	Olhar humano focado em segurança	Branca	Pull request
Testes de casos de abuso	Testes automatizados do ponto de vista do atacante	Cinza	Na integração
Fuzzing	Envia entradas malformadas ou aleatórias	Preta ou cinza	Ambiente de teste
DAST	Ataca a aplicação em execução	Preta	Ambiente de teste
Teste de invasão	Especialista simula um ataque real	Preta ou cinza	Antes de releases

Referenciais da OWASP que guiam os testes

OWASP Top 10

Os dez riscos mais críticos de aplicações web (2021).

API Security Top 10

Riscos específicos de APIs (2023), como BOLA e falhas de autenticação.

WSTG

Web Security Testing Guide: como testar cada tipo de falha.

ASVS

Lista de requisitos de segurança verificáveis, por nível.

Top 10 para LLM

Riscos de aplicações com IA generativa, como prompt injection.

No exemplo

Cada caso de teste aponta o item do Top 10 ou do API Top 10 que cobre.

Para que servem os testes de segurança

1. **Encontrar cedo**, quando corrigir é mais barato (shift-left).
2. **Garantir confidencialidade, integridade e disponibilidade** dos dados e serviços.
3. **Verificar requisitos não funcionais** e atender à LGPD, essencial com dados de estudantes.
4. **Evitar regressões:** cada falha encontrada vira teste automatizado que roda a cada commit.
5. **Gerar evidências objetivas** (relatórios) para a equipe, o cliente e auditorias.

Prós e contras na prática

Vantagens

- Reduz o custo: a falha é achada antes da produção
- Roda no CI/CD a cada commit e evita regressões
- Protege dados, reputação e conformidade com a LGPD
- Ferramentas maduras e gratuitas (ZAP, Bandit, Semgrep, pip-audit, Gitleaks)
- Gera evidências e cultura de segurança no time

Desvantagens

- Nenhuma ferramenta cobre tudo; lógica de negócio exige testes próprios
- Falsos positivos (sobretudo em SAST) consomem triagem
- DAST e pentest exigem a aplicação rodando e são mais lentos
- Pedem conhecimento para configurar e interpretar
- Teste ativo no ambiente errado causa dano; exige autorização

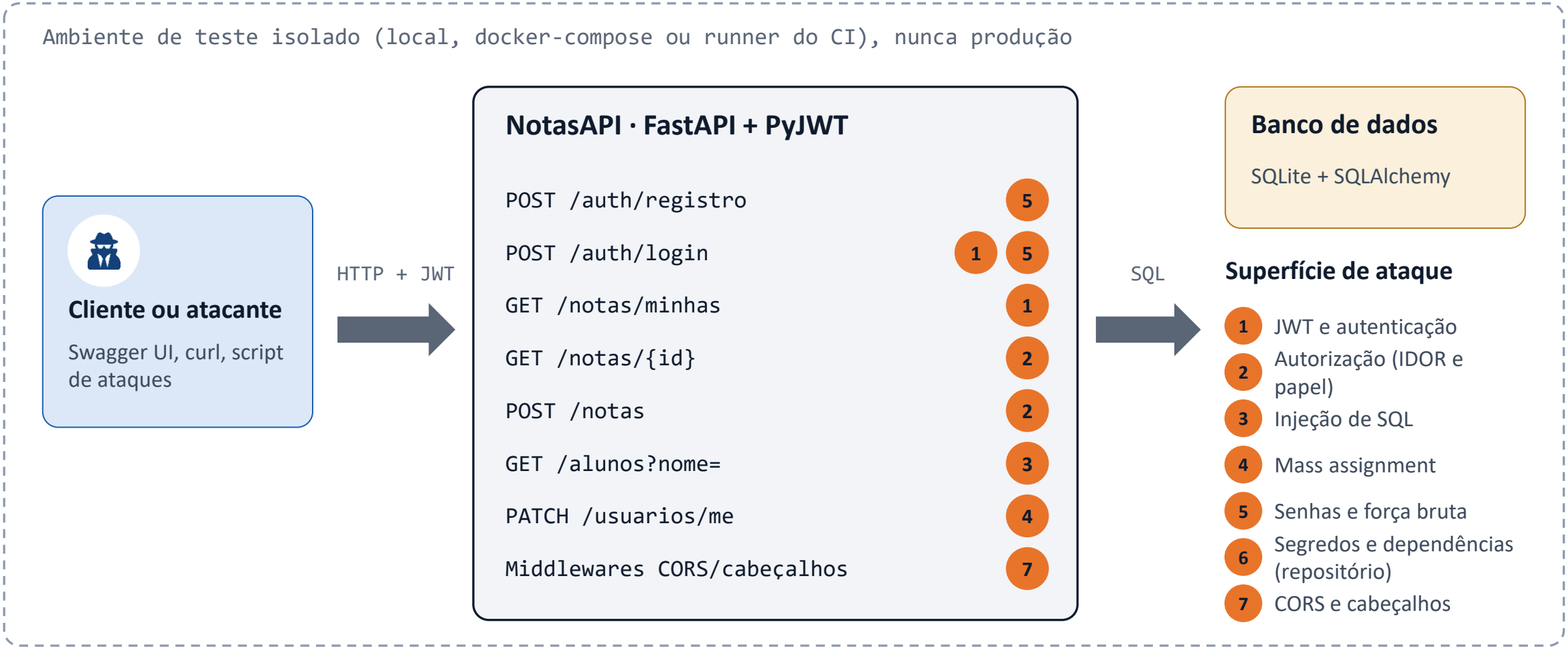
No nosso exemplo: zero falsos positivos do Bandit e do Semgrep na main; o ZAP levou alguns minutos por execução; IDOR e mass assignment só foram pegos pelos testes da equipe.

Ferramentas por categoria

Categoria	Ferramentas
SAST (código-fonte)	**Bandit**, **Semgrep**, SonarQube, CodeQL
DAST (app em execução)	**OWASP ZAP**, Burp Suite, Nikto
SCA (dependências)	**pip-audit**, OWASP Dependency-Check, Snyk, Dependabot
Segredos no repositório	**Gitleaks**, TruffleHog
Testes automatizados	**pytest** + **TestClient do FastAPI** (httpx), **PyJWT** para forjar tokens
Orquestração (CI/CD)	**GitHub Actions**
Referenciais	OWASP Top 10, API Security Top 10, WSTG, ASVS, Top 10 para LLM

Em negrito: usadas no exemplo prático.

Arquitetura e superfície de ataque



Testes em camadas no GitHub Actions



Os cinco jobs rodam **em paralelo**; qualquer achado reprova a execução.

Falhas plantadas e suas correções

#	Falha na branch vulneravel	Correção na branch main	Teste
1	Segredo fixo "segredo123"; assinatura ignorada	Segredo forte no ambiente; HS256 e exp exigidos	CT02–CT04
2	GET /notas/{id} não confere o dono (IDOR)	Só dono ou professor; 404 para os demais	CT06
2	POST /notas não confere o papel	Exige PROFESSOR, lido do banco	CT05
3	/alunos?nome= concatena texto no SQL	Consulta parametrizada pelo ORM	CT08
4	PATCH /usuarios/me aceita qualquer campo	Pydantic com extra="forbid"	CT07
5	MD5 sem salt; login sem limite	bcrypt; HTTP 429 após 5 erros	CT09, CT10
6	.env versionado; PyJWT 2.3.0 com CVEs	.env ignorado; dependências atualizadas	CT12, CT13
7	CORS *; sem cabeçalhos de segurança	CORS restrito; nosniff, CSP, X-Frame	CT11, CT15

```
git diff vulneravel main -- app/
```

Antes e depois: IDOR em GET /notas/{id}

vulneravel

```
nota = db.get(Nota, nota_id)
if nota is None:
    raise HTTPException(404)
return nota
# qualquer aluno lê qualquer nota
```

main

```
nota = db.get(Nota, nota_id)
if nota is None or (
        usuario.papel != PROFESSOR
        and nota.aluno_id != usuario.id):
    raise HTTPException(404)
return nota
```

O teste que prova a falha e protege a correção (tests/test_autorizacao.py)

```
# CT06 – a aluna Ana tenta ler a nota do Bruno
resposta = client.get(f"/notas/{nota_do_bruno['id']}", headers=auth(token_ana))
assert resposta.status_code in (403, 404)
```

Roteiro da demonstração ao vivo

1	**Subir o ambiente local** e mostrar a API em /docs	`docker compose up`
2	**Explicar a pipeline** no GitHub Actions, etapa por etapa	`aba Actions`
3	**Branch vulneravel:** ferramentas e testes falhando, com relatórios	`pytest · Artifacts`
4	**Ataques ao vivo:** Ana lê a nota do Bruno (IDOR) e vira professora (mass assignment)	`demo_ataques.py`
5	**Branch main:** pipeline aprovada e comparação antes e depois	`git diff`

Antes e depois em números

Camada / ferramenta	Branch vulneravel	Branch main
pytest (21 testes)	**16 falham**	**21 passam**
Bandit (médio ou alto)	**2 achados (SQL, MD5)**	**0**
Semgrep	**5 achados**	**0**
pip-audit	**18 avisos no PyJWT 2.3.0**	**0**
Gitleaks	**1 segredo no .env**	**0**
OWASP ZAP	**3 alertas, incl. SQLi alto**	**0 (2 informativos)**
Pipeline	**reprova**	**aprova**

16 / 21

testes falham na vulneravel

21 / 21

testes passam na main

Na vulneravel, os 5 testes que passam são o CT01 e os 4 testes de controle (uso legítimo).

O que cada técnica detectou

Falha	pytest	Bandit	Semgrep	pip-audit	Gitleaks	ZAP
Segredo JWT fraco no código	**CT02**	**baixa**	—	—	—	—
JWT sem verificação, alg none	**CT02–04**	—	**sim**	—	—	—
IDOR e aluno lançando nota	**CT05–06**	—	—	—	—	—
Mass assignment (papel)	**CT07**	—	—	—	—	—
Injeção de SQL	**CT08**	**sim**	**sim**	—	—	**sim**
MD5 sem salt	**CT10**	**sim**	**sim**	—	—	—
Força bruta no login	**CT09**	—	—	—	—	—
CORS * e cabeçalhos ausentes	**CT11, CT15**	—	**CORS**	—	—	**sim**
Chave de API no .env versionado	—	—	—	—	**sim**	—
PyJWT 2.3.0 com CVEs	—	—	—	**sim**	—	—
Erro 500 não planejado	—	—	—	—	—	**sim**

Três lições do exemplo prático



Lógica de negócio exige testes próprios

IDOR, papel, mass assignment e força bruta passaram por todas as ferramentas genéricas. Só os testes da equipe, que conhecem as regras, pegaram.



Cada ferramenta tem um ponto cego

O Gitleaks não reconheceu "segredo123" (baixa entropia) e o Bandit só deu aviso baixo. Quem provou o risco foi o CT02, com uma lista de senhas comuns.



O DAST achou o que não plantamos

O ZAP causou erro 500 ao trocar o e-mail por um já usado. Corrigimos e o achado virou o teste de regressão CT16.

Nenhuma técnica sozinha basta: segurança se testa em camadas, a cada commit.



Automatize no CI

SAST, SCA, segredos e DAST são baratos de ligar e pegam os padrões conhecidos.



Escreva casos de abuso

Testes que pensam como o atacante cobrem a lógica que as ferramentas não enxergam.



Teste com responsabilidade

Só em ambiente isolado e com autorização; nunca contra sistemas de terceiros.

Referências

ANDERSON, J. P. **Computer security technology planning study**. Bedford: U.S. Air Force Electronic Systems Division, 1972. (ESD-TR-73-51).

BANDIT. **Bandit documentation**. PyCQA, [s.d.]. Disponível em: https://bandit.readthedocs.io/. Acesso em: 26 out. 2026.

BCRYPT. **bcrypt**. Python Cryptographic Authority, [s.d.]. Disponível em: https://pypi.org/project/bcrypt/. Acesso em: 26 out. 2026.

FASTAPI. **Testing**. [s.d.]. Disponível em: https://fastapi.tiangolo.com/tutorial/testing/. Acesso em: 26 out. 2026.

FELDERER, M. et al. Security testing: a survey. **Advances in Computers**, v. 101, p. 1-51, 2016.

GITHUB. **GitHub Actions documentation**. [s.d.]. Disponível em: https://docs.github.com/actions. Acesso em: 26 out. 2026.

GITLEAKS. **Gitleaks**. [s.d.]. Disponível em: https://github.com/gitleaks/gitleaks. Acesso em: 26 out. 2026.

HOWARD, M.; LIPNER, S. **The security development lifecycle**. Redmond: Microsoft Press, 2006.

Referências

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010**: systems and software engineering - systems and software quality requirements and evaluation (SQuaRE) - product quality model. Geneva: ISO, 2023.

MCGRAW, G. **Software security**: building security in. Boston: Addison-Wesley, 2006.

MYERS, G. J.; SANDLER, C.; BADGETT, T. **The art of software testing**. 3. ed. Hoboken: John Wiley & Sons, 2011.

OWASP FOUNDATION. **OWASP API Security Top 10**. 2023. Disponível em: https://owasp.org/API-Security/. Acesso em: 26 out. 2026.

OWASP FOUNDATION. **OWASP Application Security Verification Standard**. Versão 5.0. 2025. Disponível em: https://owasp.org/www-project-application-security-verification-standard/. Acesso em: 26 out. 2026.

OWASP FOUNDATION. **OWASP Top 10 for Large Language Model Applications**. 2025. Disponível em: https://genai.owasp.org/. Acesso em: 26 out. 2026.

OWASP FOUNDATION. **OWASP Top Ten**. 2021. Disponível em: https://owasp.org/www-project-top-ten/. Acesso em: 26 out. 2026.

OWASP FOUNDATION. **OWASP Web Security Testing Guide**. Versão 4.2. 2020. Disponível em: https://owasp.org/www-project-web-security-testing-guide/. Acesso em: 26 out. 2026.

Referências

PIP-AUDIT. **pip-audit**. Python Packaging Authority, [s.d.]. Disponível em: https://github.com/pypa/pip-audit. Acesso em: 26 out. 2026.

POTTER, B.; MCGRAW, G. Software security testing. **IEEE Security & Privacy**, v. 2, n. 5, p. 81-85, 2004.

PYJWT. **PyJWT documentation**. [s.d.]. Disponível em: https://pyjwt.readthedocs.io/. Acesso em: 26 out. 2026.

PYTEST. **pytest documentation**. [s.d.]. Disponível em: https://docs.pytest.org/. Acesso em: 26 out. 2026.

SCARFONE, K. et al. **Technical guide to information security testing and assessment**. Gaithersburg: National Institute of Standards and Technology, 2008. (NIST Special Publication 800-115).

SEMGREP. **Semgrep documentation**. [s.d.]. Disponível em: https://semgrep.dev/docs/. Acesso em: 26 out. 2026.

SPAFFORD, E. H. The internet worm program: an analysis. **ACM SIGCOMM Computer Communication Review**, v. 19, n. 1, p. 17-57, 1989.

STUTTARD, D.; PINTO, M. **The web application hacker's handbook**: finding and exploiting security flaws. 2. ed. Indianapolis: Wiley, 2011.

ZAP. **ZAP documentation**. [s.d.]. Disponível em: https://www.zaproxy.org/docs/. Acesso em: 26 out. 2026.



Obrigado!
Perguntas?

Código, testes e pipeline:

`github.com/Equipe-Skyfall/Security-Testing`

Eduardo Fontes · Eduardo Kuwahara · Fábio Hiroshi · João Rossi · Kathellyn Caroline · Paulo Henrique · Victor Daniel